

Documentação do Simulador de Sistemas Operacionais

Guia Detalhado e Didático do Código Fonte

Desenvolvido para o Projeto 2026

13 de junho de 2026

Conteúdo

1	Introdução ao Projeto	2
2	Estrutura Geral dos Arquivos	2
3	Os Programas: Como eles nascem (process.c e process.h)	2
4	O Caixa do Banco: Escalonamento (scheduler.c)	3
4.1	Round-Robin (O Revezamento Justo)	3
4.2	SJF Preemptivo (Os Mais Rápidos Primeiro)	3
4.3	Prioridade Preemptiva (Os Vips)	3
5	A Mesa de Trabalho: Memória Virtual (memory.c)	4
5.1	A Política FIFO (O Primeiro a Entrar, é o Primeiro a Sair)	4
5.2	A Política LRU (<i>Least Recently Used</i>)	4
5.3	A Política Ótima (A Bola de Cristal)	4
6	Os Maestros do Espetáculo (main.c e mainwindow.cpp)	4
7	Mergulho no Código: Detalhamento Exaustivo das Funções	5
7.1	O Módulo de Processos (process.h e process.c)	5
7.1.1	process.h	5
7.1.2	process.c	5
7.2	O Módulo Escalonador (scheduler.h e scheduler.c)	5
7.2.1	scheduler.c - Funções Auxiliares	6
7.2.2	scheduler.c - Algoritmos (O Núcleo)	6
7.3	O Módulo de Memória Virtual (memory.h e memory.c)	6
7.3.1	memory.c - Gerenciamento de Quadros (Frames)	6
7.3.2	memory.c - Lógicas de Substituição (<i>Page Replacement</i>)	7
7.3.3	memory.c - Simulação de Acesso e Falhas (<i>Page Faults</i>)	7
7.4	Interação Humano-Máquina (main.c e mainwindow.cpp)	7
7.4.1	main.c (Interface Terminal de Desenvolvedor)	7
7.4.2	mainwindow.cpp (O Frontend Gráfico em Qt C++)	8
8	Conclusão Definitiva	8

1 Introdução ao Projeto

Este documento explica, de forma clara e acessível, o funcionamento do código-fonte do **Simulador de Sistemas Operacionais**. O objetivo deste projeto é imitar matematicamente como um computador real toma decisões cruciais nos bastidores, lidando com duas tarefas vitais:

1. **Decidir quem usa o processador (Escalonamento):** Como o cérebro do computador divide sua atenção entre vários programas abertos ao mesmo tempo.
2. **Gerenciar o espaço (Memória Virtual):** Como o computador lida com a falta de espaço quando tentamos abrir muitos aplicativos pesados, usando um truque chamado "Paginação" para usar o disco rígido como uma memória de mentira (virtual).

Mesmo que você não tenha experiência prévia com programação, as explicações a seguir usam analogias do dia a dia para que você compreenda como o sistema pensa e age.

2 Estrutura Geral dos Arquivos

Um programa de computador geralmente é dividido em várias partes (arquivos), assim como um livro é dividido em capítulos. Neste projeto, a divisão foi feita da seguinte forma:

- **include/** (Os Manuais): Contém arquivos que terminam em `.h`. Eles são como índices ou resumos que dizem para o sistema "quais ferramentas existem", sem detalhar como elas funcionam por dentro.
- **src/** (As Engrenagens): Contém arquivos que terminam em `.c` e `.cpp`. É aqui que o código real (a "mão na massa") está escrito.
- **resources/** (A Matéria Prima): Pastas com os dados que alimentam o sistema, como o arquivo `processos_exemplo.csv`, que diz quais programas devem ser simulados.

3 Os Programas: Como eles nascem (`process.c` e `process.h`)

Imagine que um processo no computador seja um cliente esperando para ser atendido em um banco. O arquivo `process.h` define o "crachá" desse cliente, ou seja, as informações que o sistema precisa saber sobre ele.

No código, isso é chamado de uma *estrutura* (Struct), e guarda as seguintes informações:

- **PID (Identificador):** Um número único de senha do cliente.
- **Tempo de Chegada (*Arrival Time*):** A hora exata que o cliente entrou na fila do banco.
- **Tempo de Execução (*Burst Time*):** Quanto tempo o atendimento desse cliente vai demorar no caixa (Processador).

- **Prioridade:** O cliente é um idoso com atendimento preferencial? (Valores menores significam maior prioridade).
- **Memória Necessária:** Quanta "mesa" o cliente precisa para espalhar seus documentos durante o atendimento.

O arquivo `process.c` apenas faz o trabalho braçal de ler o arquivo de planilhas (`.csv`) e criar os crachás de todos os clientes, colocando-os em uma fila para o sistema atender.

4 O Caixa do Banco: Escalonamento (`scheduler.c`)

O arquivo `scheduler.c` é onde a mágica das filas acontece. Ele funciona como o gerente do banco que decide qual caixa vai atender qual cliente. Como o nosso computador só tem um caixa (um processador único), ele precisa revezar. O sistema implementa três estratégias (algoritmos) diferentes para fazer isso:

4.1 Round-Robin (O Revezamento Justo)

Nesta estratégia, o gerente diz: *"Todo mundo vai ser atendido por, no máximo, 4 minutos de cada vez"*. Se o seu problema demora 10 minutos, o caixa te atende por 4, pede para você ir para o final da fila, chama o próximo, e você volta depois.

- **Como funciona no código:** O código cria uma fila circular (uma roda gigante). Ele pega o primeiro da fila, roda o relógio até o limite (o *Quantum*). Se o processo terminou, ele é removido. Se não, o código literalmente copia o processo para o final da fila e repete o processo até esvaziar a sala.

4.2 SJF Preemptivo (Os Mais Rápidos Primeiro)

Imagine que você está na fila do mercado com um carrinho cheio, e chega alguém atrás com apenas 1 garrafa de água. O algoritmo *Shortest Job First* (SJF) permite que a pessoa com menos itens passe na frente. E pior (ou melhor), se você já está no caixa passando as compras e a pessoa chega, o caixa para de te atender, guarda suas sacolas e atende a pessoa rápida primeiro.

- **Como funciona no código:** A cada segundo (tempo), o código varre a lista de todos os processos que já chegaram. Ele procura aquele que tem o **menor tempo restante**. O processo com a tarefa mais curta assume o controle. Isso exige o cálculo constante a cada instante de tempo.

4.3 Prioridade Preemptiva (Os Vips)

Parecido com o anterior, mas o tamanho do trabalho não importa. O que importa é a "Carteirinha Vip".

- **Como funciona no código:** Novamente, a cada milissegundo o sistema verifica a lista. Ele escolhe o processo que tem a melhor prioridade (matematicamente, o menor número). Se um processo super prioritário nascer no meio do nada, ele interrompe o que está sendo executado e rouba a CPU para si.

5 A Mesa de Trabalho: Memória Virtual (`memory.c`)

A memória RAM do computador é muito rápida, mas muito pequena (como a sua mesa de trabalho). O disco rígido (HD/SSD) é enorme, mas muito lento (como o arquivo no fundo da sala). O `memory.c` simula a tática de trazer para a mesa apenas os papéis que você precisa usar agora (Páginas), guardando o resto no arquivo.

Quando o sistema precisa de um papel que não está na mesa, acontece um erro chamado **Page Fault** (Falha de Página). Você precisa pausar tudo, ir até o arquivo, pegar o papel e voltar. O objetivo do sistema é minimizar a quantidade de vezes que você levanta da cadeira. Se a mesa lotar, qual papel você devolve pro arquivo? Temos 3 políticas para isso:

5.1 A Política FIFO (O Primeiro a Entrar, é o Primeiro a Sair)

A lógica é simples: o papel que está na mesa há mais tempo é o escolhido para voltar pro arquivo. O código usa uma contagem e a posição sequencial para sempre apontar para o papel mais "velho".

5.2 A Política LRU (*Least Recently Used*)

O LRU (Menos Recentemente Usado) é mais esperto. Ele pensa: *"Vou jogar fora o papel que faz mais tempo que eu não toco"*.

- **Como o código faz isso:** Cada página na memória guarda a data/hora (o *timestamp*) em que ela foi lida pela última vez. O algoritmo varre todas as páginas e encontra a que tem o tempo mais antigo.

5.3 A Política Ótima (A Bola de Cristal)

Este é o algoritmo perfeito. Ele é impossível na vida real porque exige prever o futuro. Ele se pergunta: *"Qual desses papéis na mesa eu vou demorar mais para precisar novamente?"*.

- **Como o código faz isso (Nossa Invenção):** Como estamos em uma simulação que nós mesmos construímos, o nosso simulador *conhece* o futuro. O simulador constrói uma fila do que o processo fará. Quando a memória lota, o código vasculha essa fila futurística e busca qual dos itens na memória demorará mais a ser invocado, removendo-o. Este algoritmo recentemente passou por uma pesada otimização no nosso sistema para que rodasse dezenas de milhares de vezes mais rápido usando checagens sequenciais.

6 Os Maestros do Espetáculo (`main.c` e `mainwindow.cpp`)

Toda essa lógica não tem utilidade se o humano não a visualizar.

- O `main.c` é feito para uso em **Terminal (Tela Preta)**. Ele é focado em rapidez, rodando os cálculos de trás das cortinas sem se preocupar em pintar telas bonitas.

- O `mainwindow.cpp` é o **Gerenciador Visual (Interface Gráfica)**. Desenvolvido usando a tecnologia *Qt*, ele possui painéis, botões e caixas de seleção. Quando você aperta "Simular", ele conversa com nossos arquivos C, entrega as planilhas e aguarda os cálculos. Ao receber de volta, ele desenha quadrados coloridos na tela que formam o *Gráfico de Linha do Tempo* (Gráfico de Gantt).

7 Mergulho no Código: Detalhamento Exaustivo das Funções

Esta seção dissectiona profissionalmente cada arquivo de código (`.h`, `.c`, `.cpp`) e todas as suas funções. Explicaremos qual a responsabilidade exata de cada pedaço de software no sistema.

7.1 O Módulo de Processos (`process.h` e `process.c`)

Responsável por interpretar a entrada de dados (CSV) e modelar a estrutura fundamental da simulação, dizendo "quem é" cada processo.

7.1.1 `process.h`

Define os esqueletos das informações (Estruturas de Dados).

- **ProcessState (Enumeração):** Uma lista de etiquetas. Define os estados possíveis de um processo: `STATE_NEW` (Novo), `STATE_READY` (Pronto), `STATE_RUNNING` (Rodando), `STATE_WAITING` (Esperando), e `STATE_TERMINATED` (Finalizado).
- **Process (Struct):** A espinha dorsal. Uma "caixa" de informações que armazena `pid`, `arrival_time`, `burst_time`, `remaining_time`, `priority`, e `memory_needed`. Também guarda as métricas finais após a simulação acabar (`waiting_time`, `turnaround_time`).

7.1.2 `process.c`

A fábrica que lê as informações e monta os processos de fato.

- `load_processes_from_csv(filepath, procs, max)`: Função de Leitura de Arquivo (*I/O*). Ela abre o arquivo CSV usando a função `fopen`, ignora o cabeçalho e lê linha por linha usando `fgets`. A função `strtok` é utilizada para "fatiar" as informações da linha divididas por vírgulas, e a função `atoi` converte os textos fatiados em números matemáticos reais. O resultado é despejado no vetor `procs`.
- `print_processes(procs, n)`: Uma função auxiliar (*Helper*). Usada por nós, desenvolvedores, para imprimir no console uma tabela bonita em texto e verificar se a conversão feita pela função acima ocorreu perfeitamente.

7.2 O Módulo Escalonador (`scheduler.h` e `scheduler.c`)

O maestro do processador. O arquivo `scheduler.h` apenas dita as regras e estruturas para guardar resultados (`TimelineEntry` e `SchedulerResult`), enquanto o `scheduler.c` implementa as lógicas profundas.

7.2.1 scheduler.c - Funções Auxiliares

- `add_timeline(r, pid, t_start, t_end)`: Responsável por "filmar" a execução. Cada vez que um processo entra ou sai do processador, essa função anota quem era o processo (`pid`) e de que momento até qual momento ele rodou (`t_start` até `t_end`). Isso será vital para a Interface Gráfica desenhar as cores depois.
- `compute_metrics(procs, n, r)`: Função estritamente matemática. Calcula o tempo de *Turnaround* (Tempo de Conclusão - Tempo de Chegada), o *Waiting Time* (Espera, deduzindo o tempo gasto trabalhando) e o *Response Time* (Resposta do clique à primeira ação). Ela gera as médias estatísticas oficiais da simulação.

7.2.2 scheduler.c - Algoritmos (O Núcleo)

- `sched_round_robin(procs, n, quantum, r)`: O Revezamento. Implementa uma Fila Circular (*Queue*). Usa as variáveis numéricas `head` e `tail` para navegar num array funcionando como uma roda gigante. Retira um processo, roda por um tempo estritamente limitado à variável `quantum`, e o devolve à fila final com o comando `ENQUEUE` caso a tarefa ainda não esteja 100% concluída.
- `sched_sjf_preemptive(procs, n, r)`: *Shortest Remaining Time First* (SRTF). Possui um loop infinito enquanto existirem processos não terminados. A cada instante de milissegundo do tempo, ele varre todos os processos (`best = i`) e seleciona estritamente o de menor `remaining_time`. Se no meio do processamento surgir um outro processo mais rápido ainda, ocorre a famosa "preempção" (roubo da CPU).
- `sched_priority_preemptive(procs, n, r)`: Funciona como um gêmeo quase idêntico ao SJF. Porém, no laço de repetição, a condição de vitória e tomada do processador é baseada em quem tem a menor `priority` (onde número 1 tem prioridade sobre o número 3).
- `run_scheduler(type, ...)`: O grande "Despachante" (*Dispatcher*). O usuário clica na interface escolhendo um nome, a interface manda um código, e a função realiza um `switch case` para engatilhar um dos 3 algoritmos explicados acima.

7.3 O Módulo de Memória Virtual (`memory.h` e `memory.c`)

Este é o módulo mais complexo matematicamente e em engenharia de sistemas do projeto. Ele cuida do armazenamento e descarte de quadros (*Paging*).

7.3.1 memory.c - Gerenciamento de Quadros (Frames)

- `memory_init(mem, physical, virtual)`: A função criadora. Aloca os "quadros físicos" (frames) de memória dinamicamente usando a função `malloc` de acordo com a conversão de MB para KB da RAM requerida pelo usuário (ex: 256MB). Inicialmente, limpa todos com o valor de -1 (que significa "Vazio").
- `find_free_frame(mem)`: Vasculha o vetor colossal da RAM procurando os ditos espaços com -1. Retorna o índice do espaço limpo ou avisa, devolvendo o próprio -1, que a mesa de trabalho física do computador já lotou e alguém precisará ser ejetado para liberar espaço.

- `memory_free_process(mem, pt, pid)`: Quando um processo avisa ao Escalonador que encerrou seus trabalhos, esta função varre a tabela do processo. Onde existir uma página na memória dele assinalada com "Válido", a função apaga esse pedaço na RAM, devolvendo os recursos ao computador.

7.3.2 `memory.c` - Lógicas de Substituição (*Page Replacement*)

Quando `find_free_frame` relata que não há mais espaço, o sistema engatilha uma destas funções candidatas a *vítima*:

- `victim_fifo(mem, pt)`: Algoritmo simples que usa um ponteiro assinalado como `victim_hint`. Ele avança o ponteiro de forma circular de 1 em 1 para garantir que a página expulsa, matematicamente, seja a mais "antiga" na fila desde que a simulação rodou.
- `victim_lru(mem, pt)`: Na nossa implementação para varreduras simples, usa o conceito temporal e posicional para identificar a página "menos tocada recentemente", funcionando ativamente em conjunto com o registro de acesso.
- `victim_optimal(pt, future_refs, future_len, current_pos)`: O milagre Otimizado da previsão. Esta função varre um vetor profético (`future_refs`) usando marcadores para enxergar de antemão quais páginas o aplicativo exigirá nos próximos segundos. Ele intercepta e ejeta a página `last_page` com cirúrgica precisão, que será a folha que tardará mais a ser necessária.

7.3.3 `memory.c` - Simulação de Acesso e Falhas (*Page Faults*)

- `memory_access_page(...)`: O guarda de segurança. Toda requisição passa por ele. Tenta ler um pedaço da memória; se já o possuir (`valid == 1`), apenas sinaliza sucesso (*Page Hit*). Se não tiver (*Page Fault*), procura espaço e rouba a vaga de quem for selecionado como vítima.
- `memory_load_process(...)`: O Gerador de Stress. Emulações cruas apenas alocam tudo 1 vez. Esta função constrói um array de requisições de 3 vezes o tamanho do processo com fatias randomizadas (por `rand()`) imitando a desorganização que ocorre com os programas reais na RAM do seu Windows ou Mac, causando falhas de páginas reais.

7.4 Interação Humano-Máquina (`main.c` e `mainwindow.cpp`)

A alma lógica existe, mas é preciso dar-lhe um rosto. Dividimos isso em duas vertentes.

7.4.1 `main.c` (Interface Terminal de Desenvolvedor)

O arquivo mestre compilado com a instrução de atalho `make cli`. Ele empacota toda as invocações das bibliotecas descritas acima e formata as saídas estatísticas na "Tela Preta" com a diretiva `printf`. Sendo livre do gigantesco peso de interfaces gráficas, ele serve ativamente para rodarmos o simulador com alto desempenho para checagem rápida de depuração em *Debugging* de código C.

7.4.2 `mainwindow.cpp` (O Frontend Gráfico em Qt C++)

O rosto visual deslumbrante e moderno escrito no paradigma Orientado a Objetos (C++ e Biblioteca Qt).

- `TimelineWidget::paintEvent()`: A classe artística do simulador de gantt. Usa a biblioteca `QPainter` para ler os registros numéricos guardados pela `add_timeline()`. Ela preenche os pixels da tela convertendo números de tempo em posições X no monitor, pintando os blocos `fillRect` com uma paleta de 8 cores para diferenciar cada processo.
- `MainWindow::buildUI()`: O construtor civil do projeto. Invoca o *Toolkit UI* arranjando `QVBoxLayout` e `QGridLayout` no grid. Insere interações de mouse em `QPushButton` e numéricas em caixas giratórias `QSpinBox`.
- `MainWindow::onSimulate()`: O sinal de Ignição. Ativada no evento *click()* do botão, recolhe todos as diretivas gráficas pedidas e orquestra uma viagem de ida e volta para as lógicas matemáticas do módulo em C puro. Pega as *Page Faults* e as injeta de volta nas `QLabel` de resultados, gerando a interface maravilhosa ao qual estamos acostumados.

8 Conclusão Definitiva

Com todos estes detalhes e pormenores, compreendemos perfeitamente a profundidade estrutural deste Simulador de Sistemas. A qualidade de todo este maquinário computacional baseia-se em um pilar inquebrável da Engenharia de Software: **o Baixo Acoplamento**. As equações estatísticas de Memória e Escalonamento estão cegas aos detalhes visuais, guardadas confiavelmente em C. O Simulador visual apenas empurra dados e recebe respostas prontas. O formato modular propicia que nosso simulador performe cálculos tridimensionais avançados na mesma facilidade que rodaria um cálculo de um programa escolar, resultando em um sistema maduro, incrivelmente rápido e estável, preparado para emular as raízes de qualquer Sistema Operacional da vida real.